



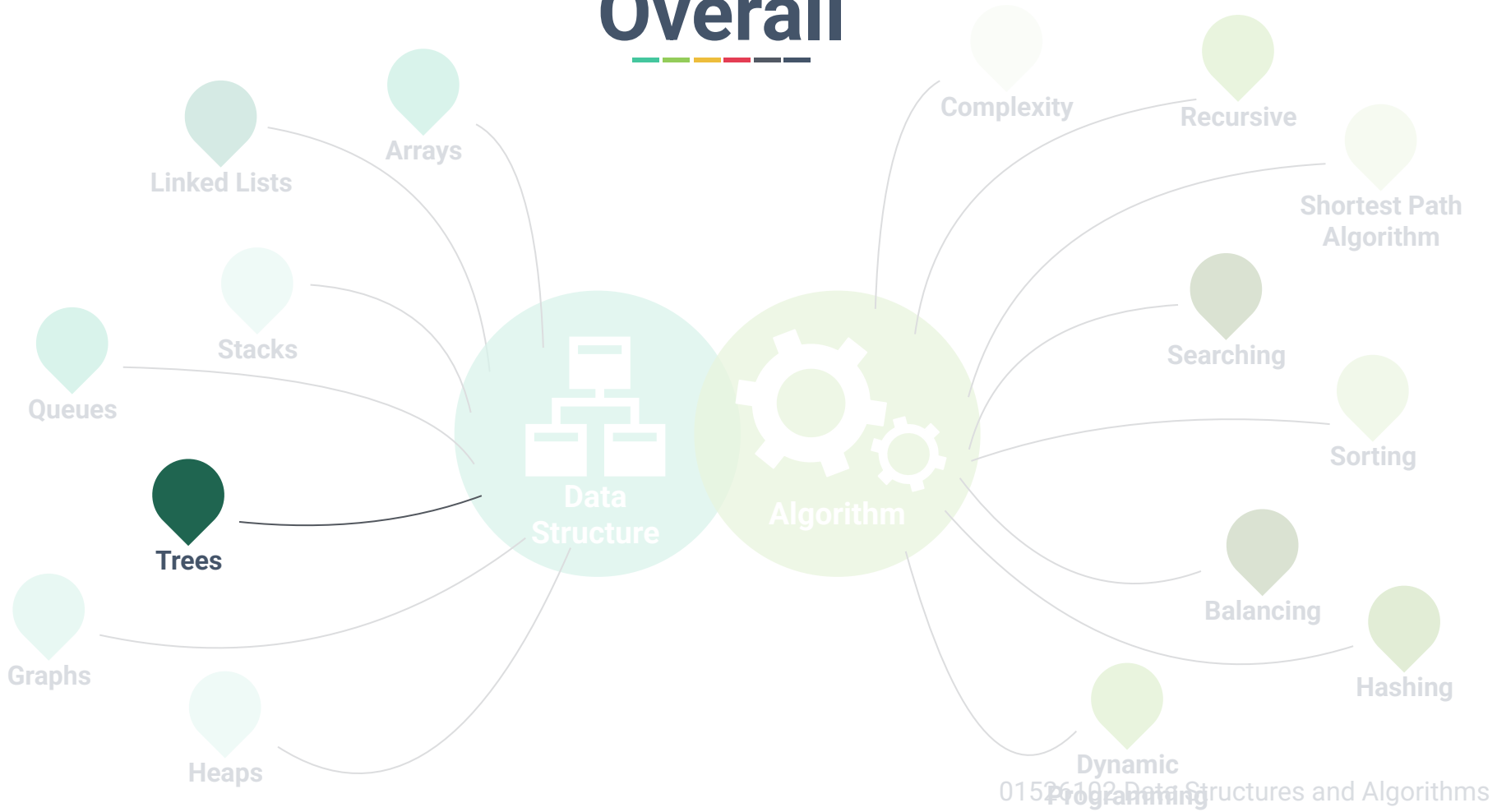
Chapter 7: Trees



Dr. Sirasit Lochanachit



Overall





Abstract Data Type



Linear:

- Arrays, Stacks, Queues, and Linked Lists
- Operations: push, pop, enqueue, dequeue
- Algorithms: Searching and sorting, insertion, deletion

Non-Linear:

- Trees and Graphs
- Algorithms: Traversal, Insertion and Deletion, Balancing, and etc.



Today's Outline



General Trees:

- Definition and examples
- Elements of Tree Structure

Binary Trees:

- Definition, properties, and types

Tree traversal algorithms:

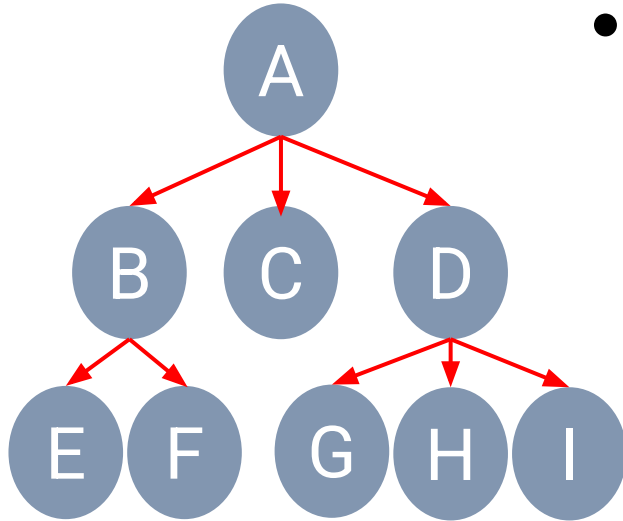
- Depth-first Traversal (Preorder, postorder, and inorder)
- Breadth-first Traversal



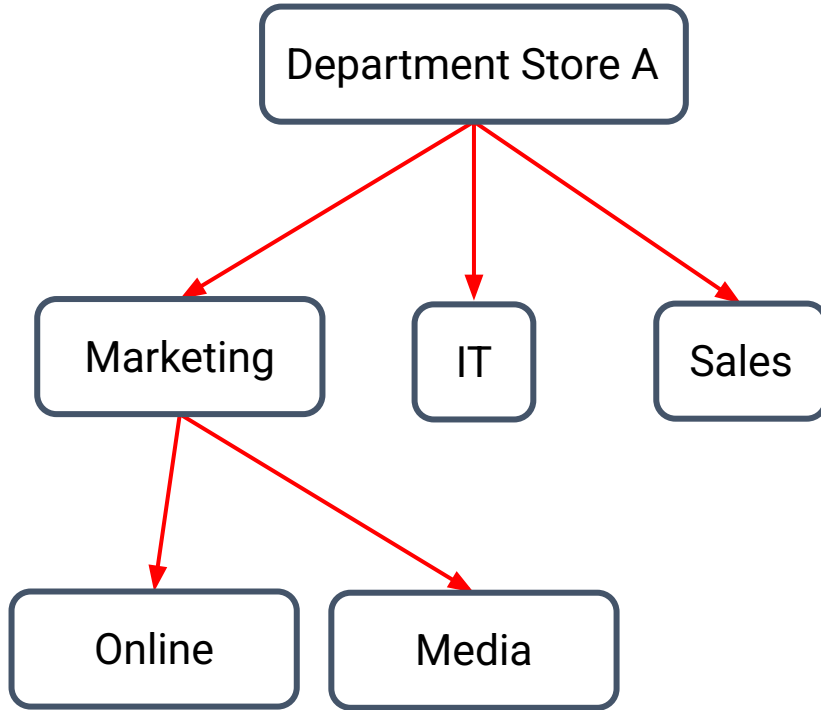
What is a Tree?



- A **tree** stores elements in a hierarchical structure.



Tree Example

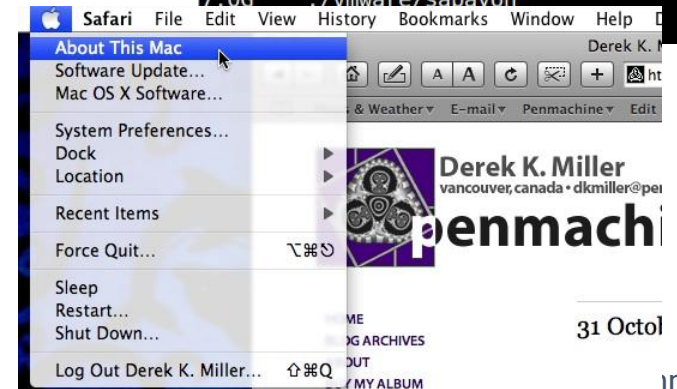


Tree Applications



- Tree represents natural organisation for data.
- Tree structure has been used widely in
 - File systems (Directory)
 - Graphical User Interfaces
 - Databases (Sub-categories, etc.)
 - Websites

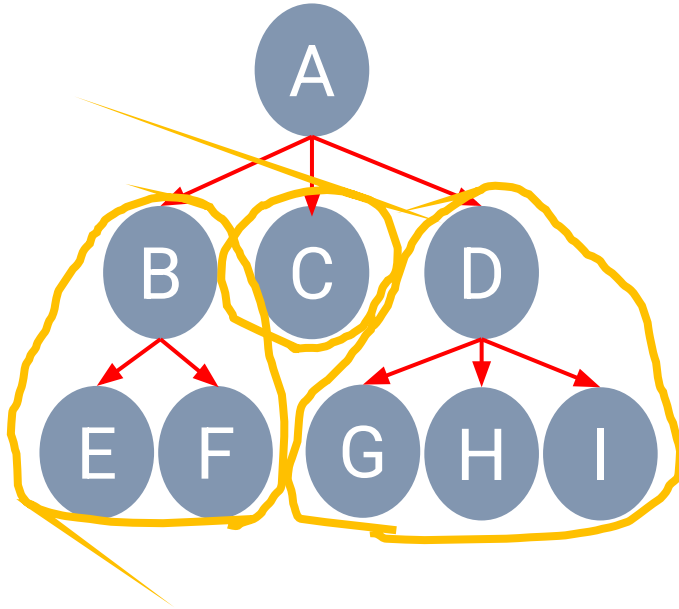
```
$ du -Sh | sort -rh | head -n 15
35G    ./vmware/iso
19G    ./vmware/ubuntu32
19G    ./Downloads
12G    ./vmware/xubuntu
12G    ./vmware/ubuntu13.10
12G    ./vmware/centos6.4
11G    ./vmware/ubuntu
11G    ./vmware/kfedora
9.8G   ./vmware/node
9.5G   ./vmware/fedora
7.7G   ./vmware/ubuntu13.04
7.6G   ./vmware/centos_server
7.4G   ./vmware/kdebian
7.1G   ./vmware/pclinuxos
7.0G   ./vmware/sabayon
```



Formal Definition

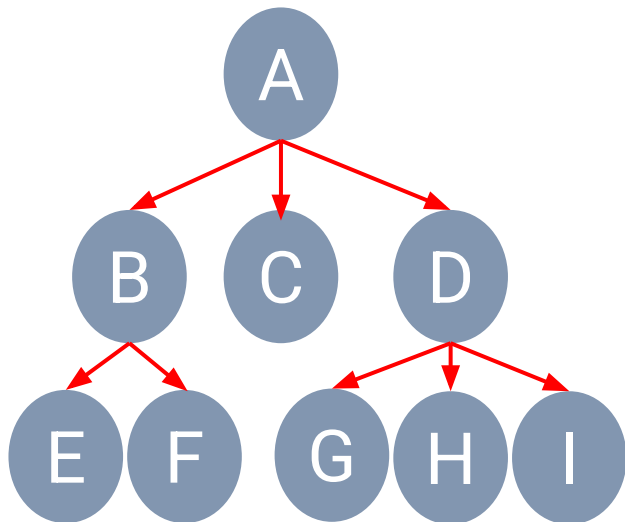


- A **tree** is a collection of nodes that store elements with a **parent-child** relationship.





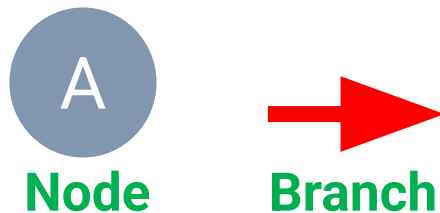
Formal Definition



- **Root**
- **Edge**
- **Child**
- **Siblings**
- **Internal** or **parent** node
- **External** or **leaf** node
- **Ancestor**
- **Descendant**



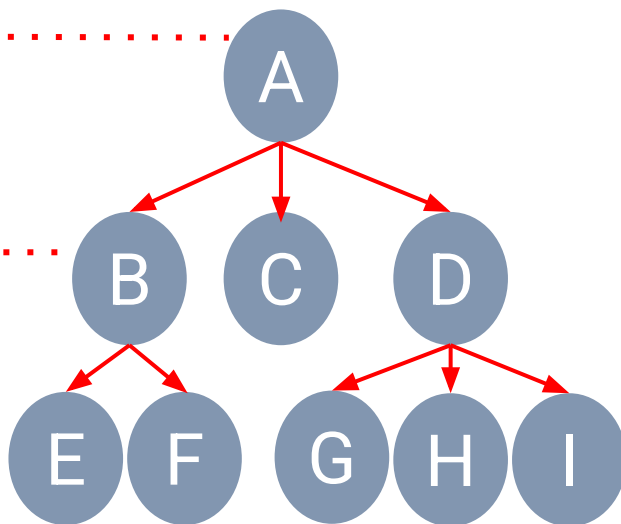
Basic Elements of Tree Structure



Level 0

Level 1

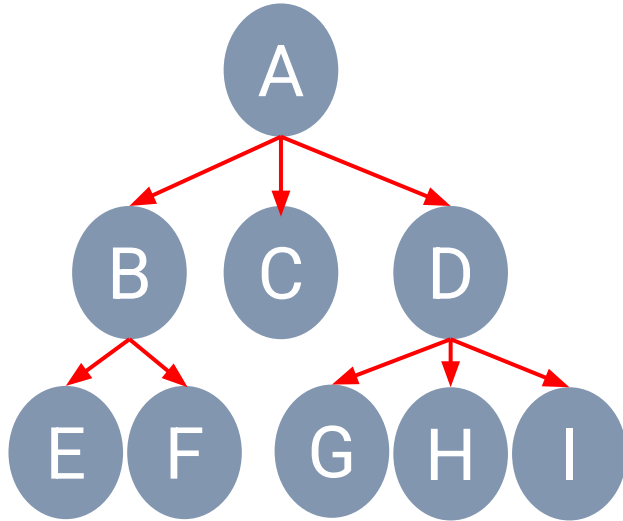
Level 2



- **Root** Node: *A*
- **Parents**: *A*, *B*, and *D*
- **Children**: *B*, *E*, *F*, *C*, *D*, *G*, *H* and *I*
- **Siblings**: {*B*, *C*, *D*}, {*E*, *F*}, {*G*, *H*, *I*}
- **Leaf** Nodes: *C*, *E*, *F*, *G*, *H* and *I*
- Degree of Tree is 8
- Degree of Node D is 3
- Height of tree is 3
- Depth of node D is 1



Other Implementation of Trees

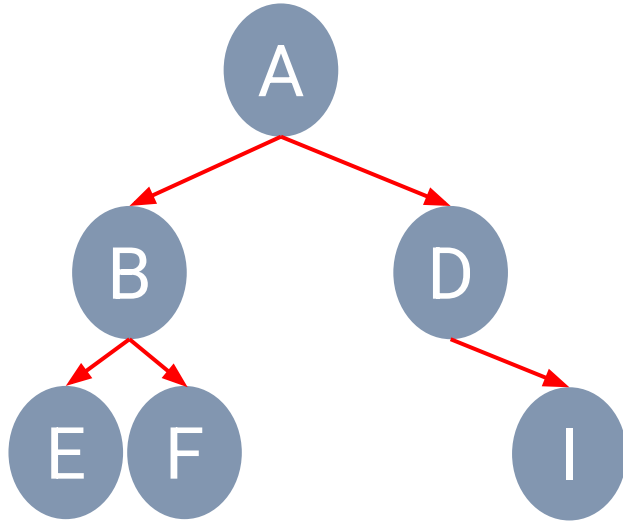


Linked List Form

$A\{ B\{E, F\}, C, D\{G, H, I\} \}$

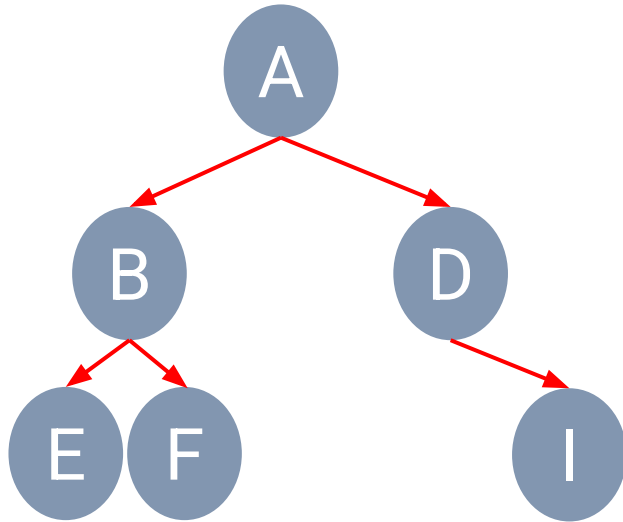
Set/List Form

Binary Trees



- A **binary tree** is a tree in which any node can have only two children at maximum.
- Each child node is designated as being either a **left child** or **right child**.

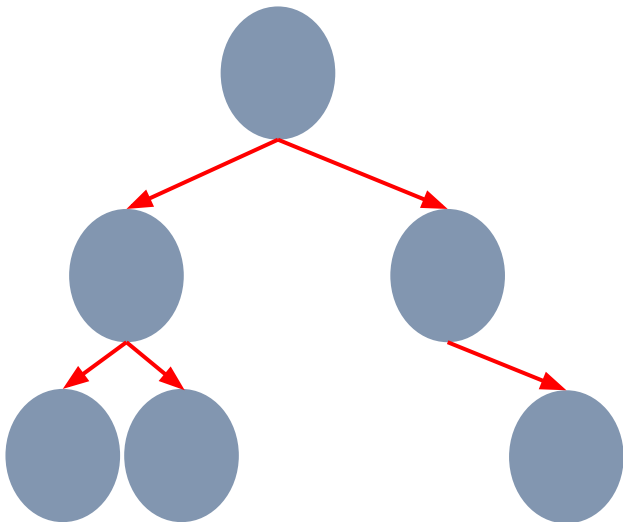
Recursive Binary Trees



- A **binary tree** is either empty or consists of:
 - A root node that stores an element
 - A binary left subtree (possibly empty)
 - A binary right subtree (possibly empty)



Binary Trees' Properties



- A **Degree** of node is a number of edges/subtrees connected to the node.
- A **Depth** of a node is a length of the path to its root (i.e. number of edges from root the node).
 - d denotes the depth/level of a node
- **Height** of tree is a number of levels starting from the root node (i.e. number of edges from the node to the deepest leaf).
 - h denotes the height of tree

Binary Trees' Properties



Number of Nodes
In each level

Level 0

1

Level 1

2

Level 2

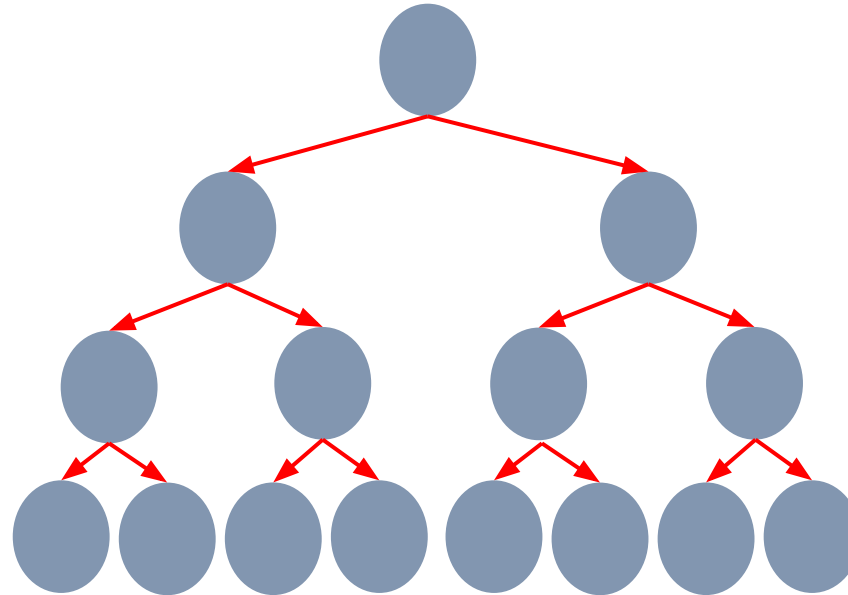
4

Level 3

8

Level d

2^d

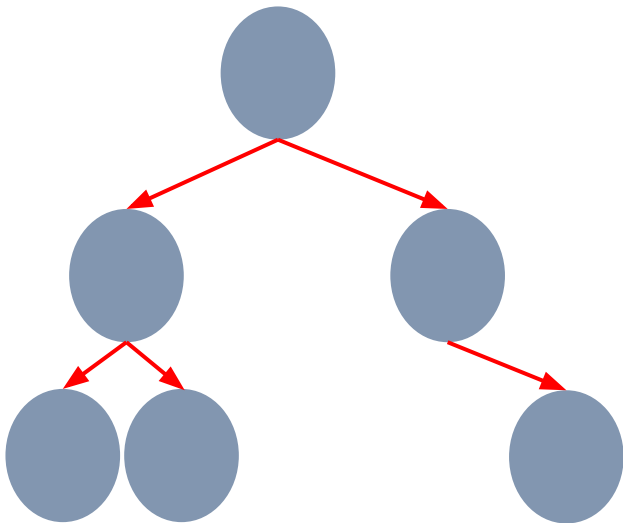




Binary Trees' Properties



Suppose n is the total number of nodes in a tree and h is the number of height of tree.



- **Maximum height** of binary trees, $h_{\max} =$
- **Minimum height** of binary trees, $h_{\min} =$
- **Minimum number** of nodes in a tree, $n_{\min} =$
- **Maximum number** of nodes in a tree, $n_{\max} =$

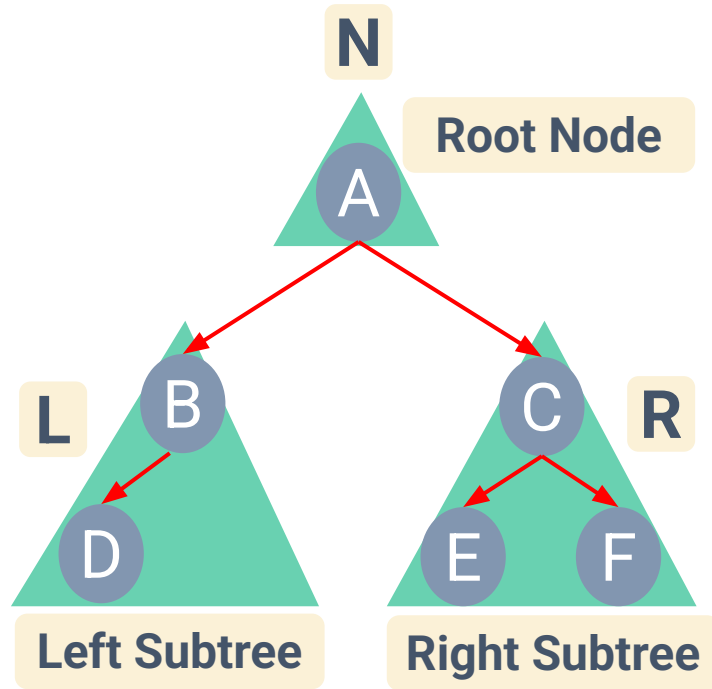


Binary Tree Traversal



- A **traversal** of a tree is a method to access all the tree nodes.
- Two main approaches: Depth-first and Breadth-first
 - Depth-first
 - Preorder traversal
 - Postorder traversal
 - Inorder traversal
 - Breadth-first - visit all the nodes at depth d before moving to depth $d+1$.

Depth-first Traversal



- **Pre**order Traversal

N L R

- **In**order Traversal

L N R

- **Post**order Traversal

L R N

Preorder Traversal

- Root -> Left subtree -> Right subtree

Algorithm preOrder(root):

if (root is not null):

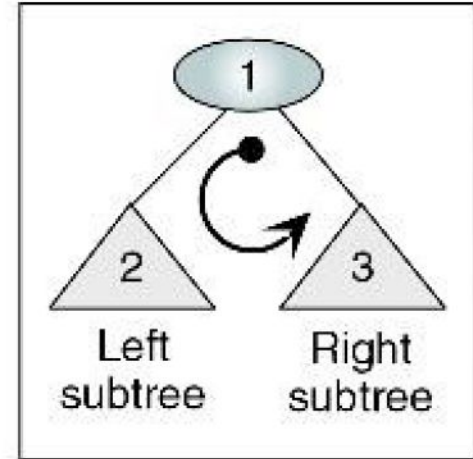
 process(root)

 preOrder(leftSubtree)

 preOrder(rightSubtree)

end if

end preOrder

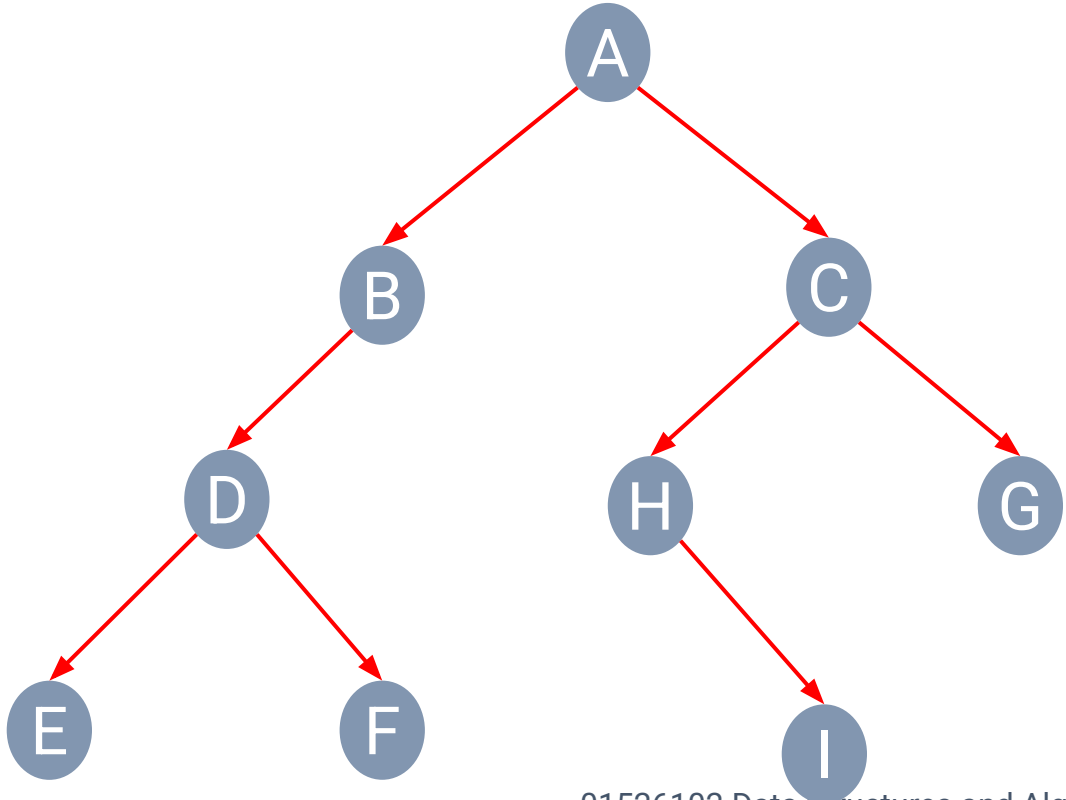


Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**



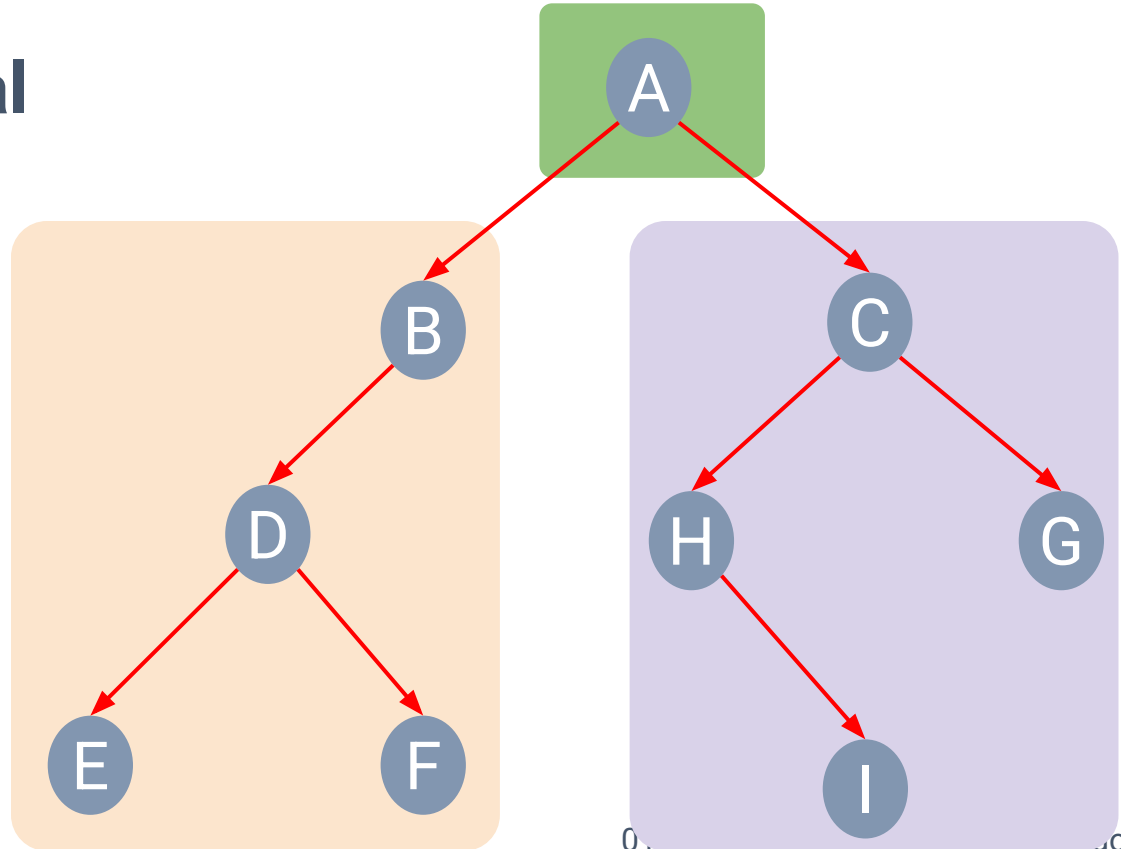
Preorder Traversal

- **Preorder Traversal**

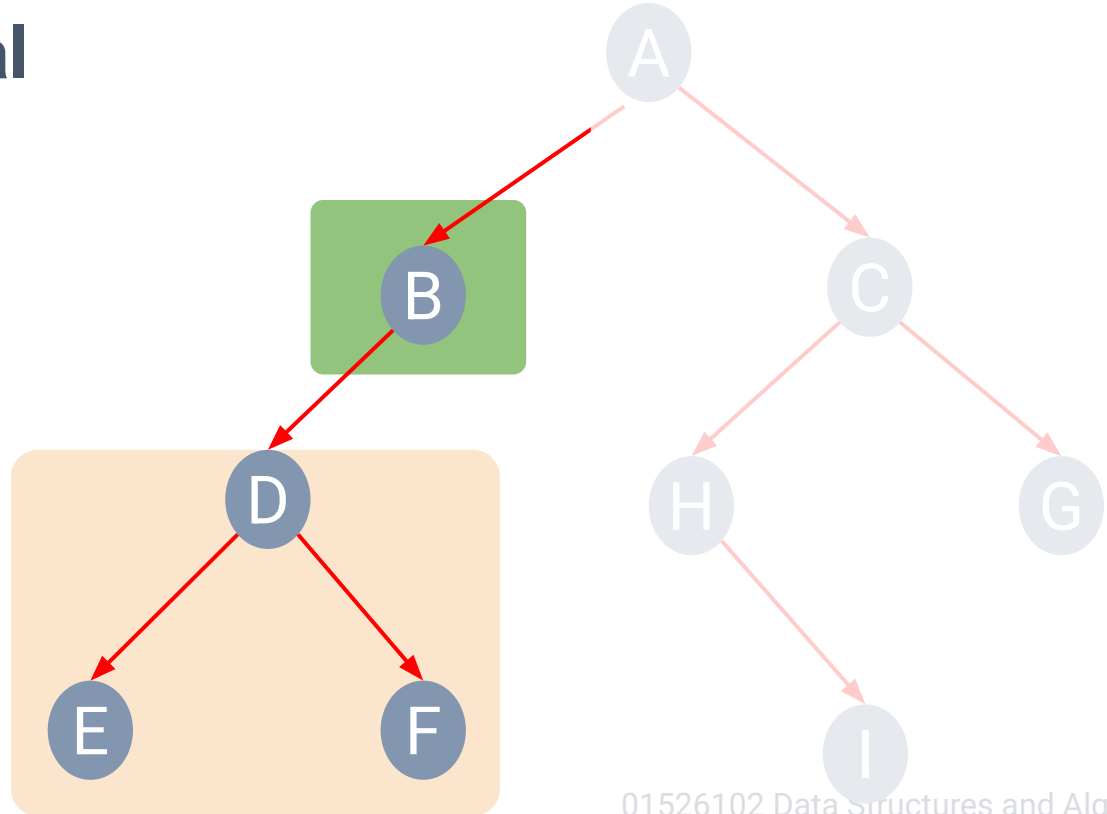
N **L** **R**

- **Visited nodes**

A



A B



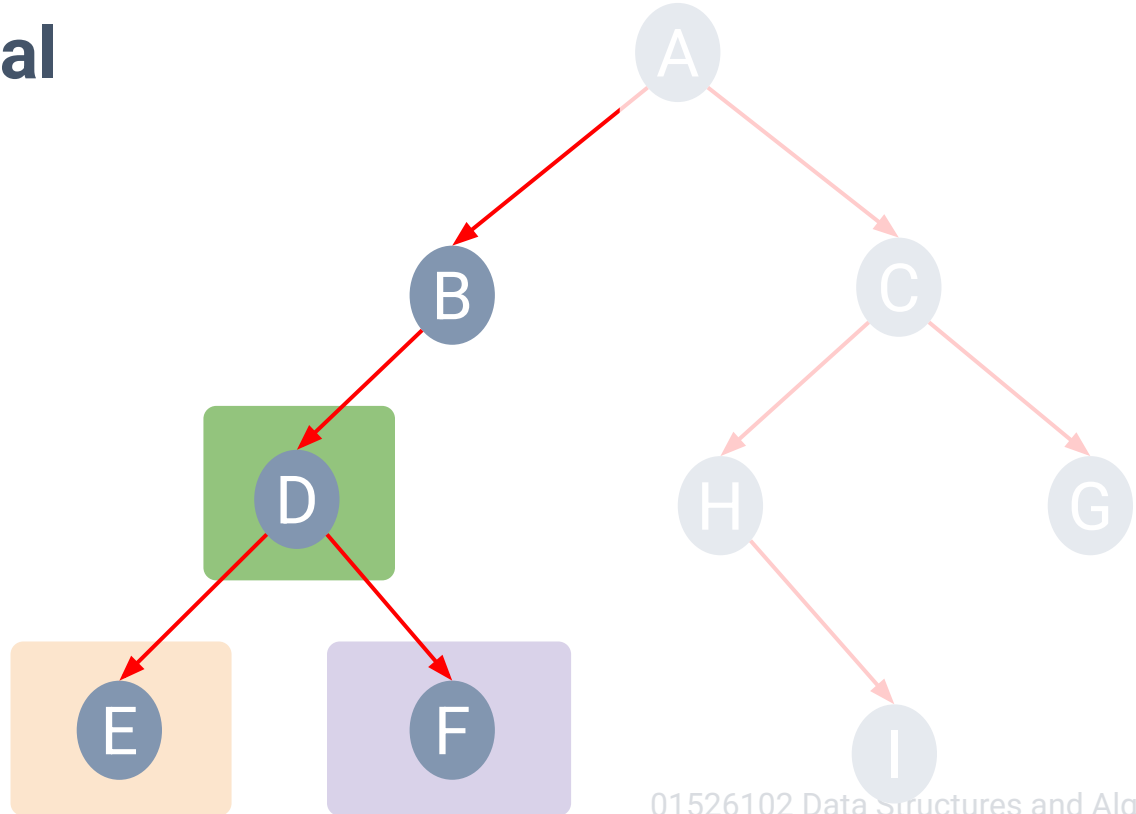
Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**

A **B** **D** **E** **F**



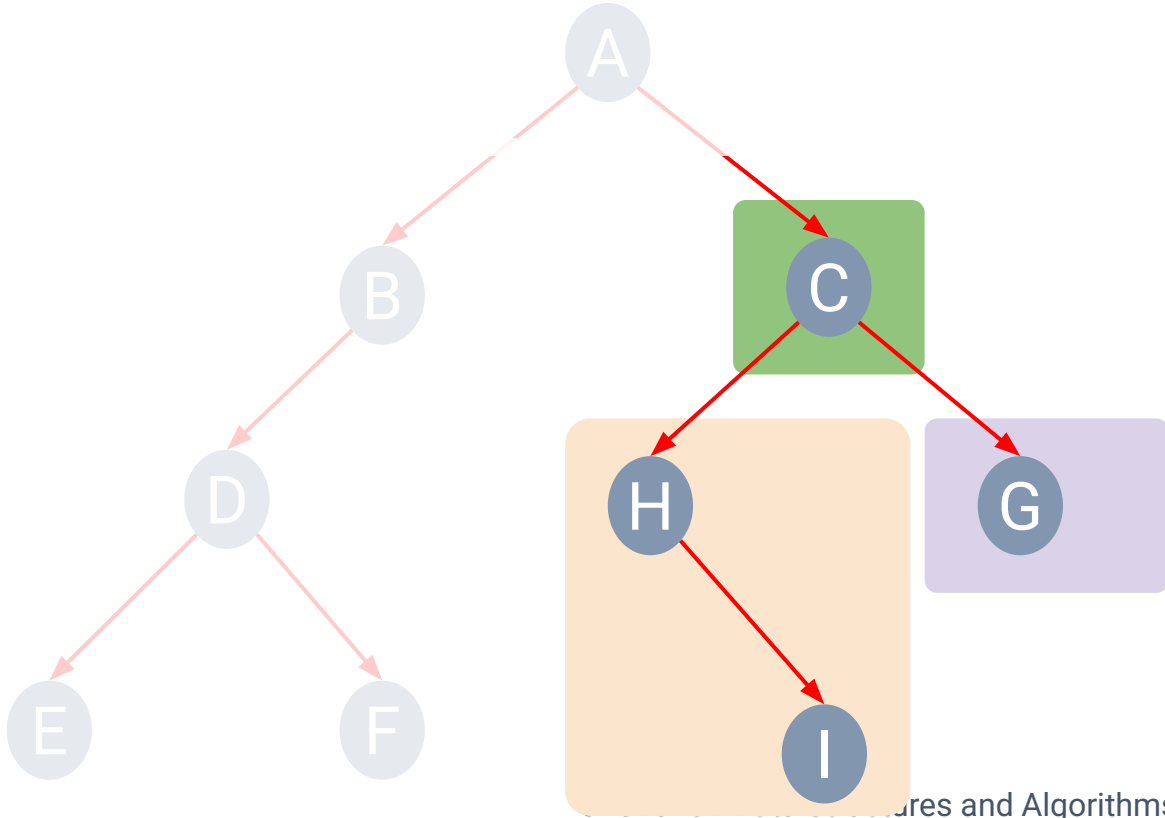
Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**

A B D E F



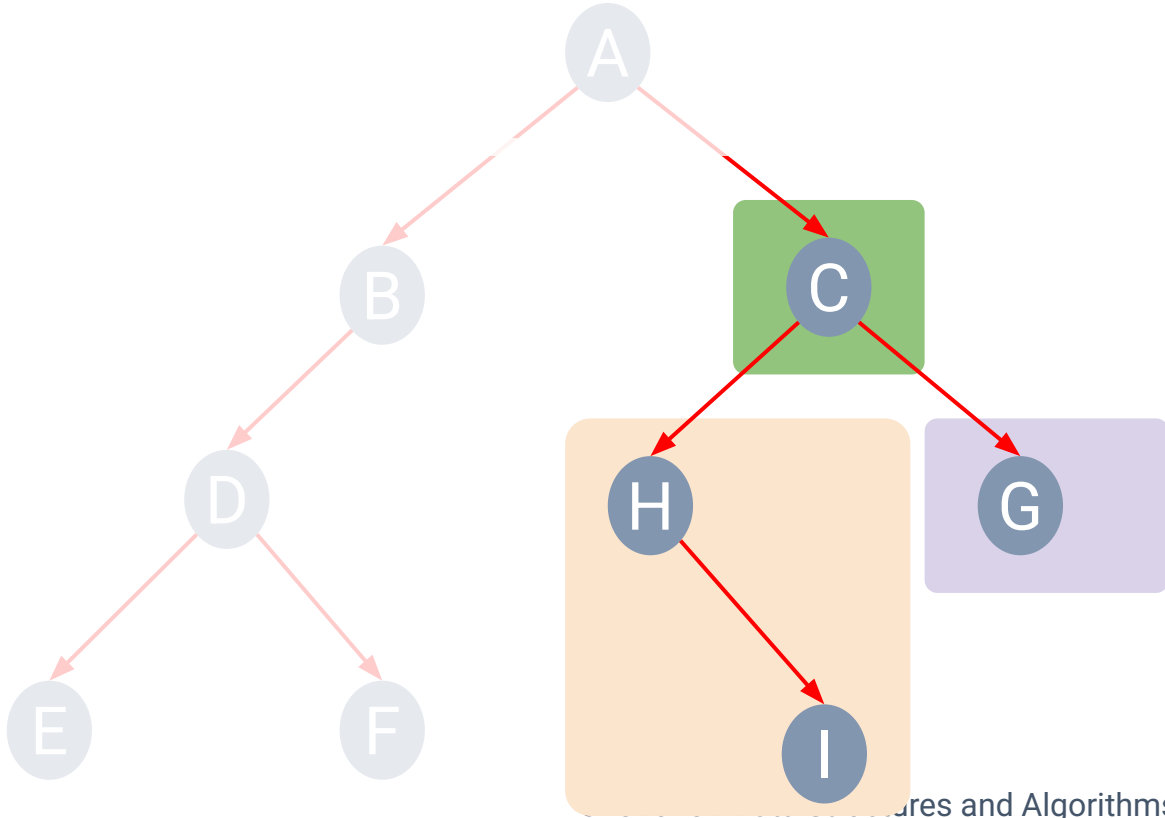
Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**

A B D E F
C



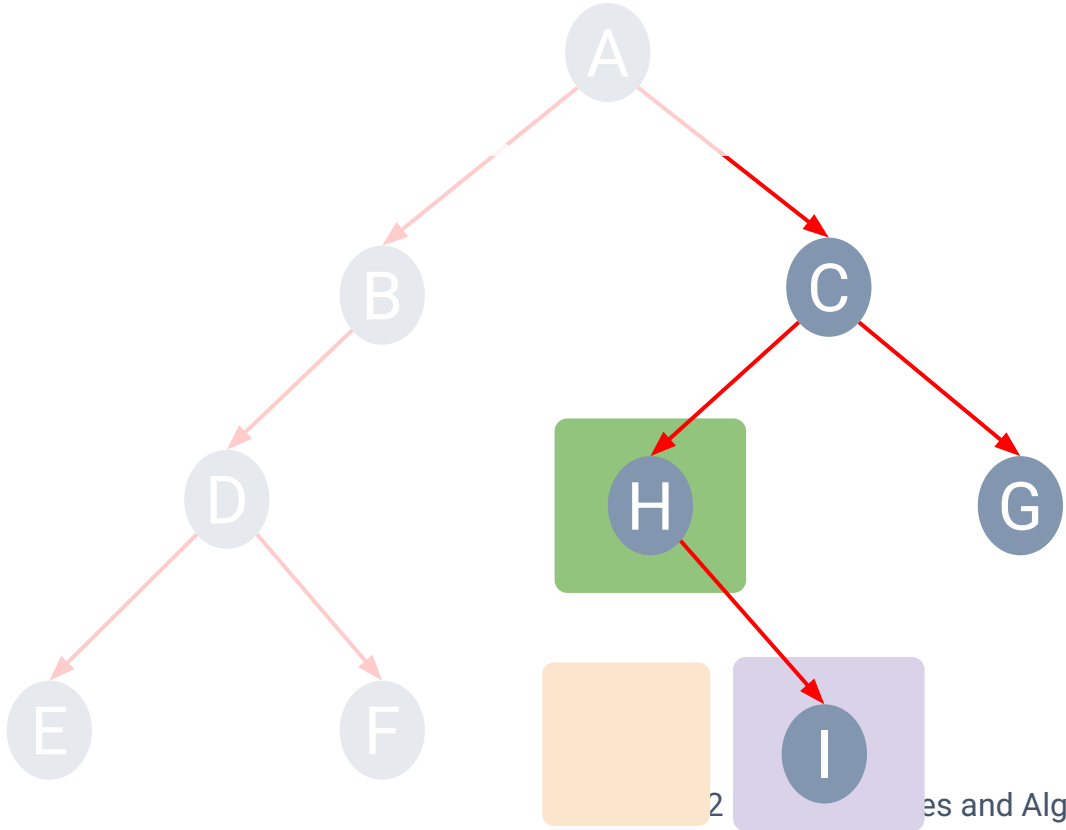
Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**

A B D E F
C H I



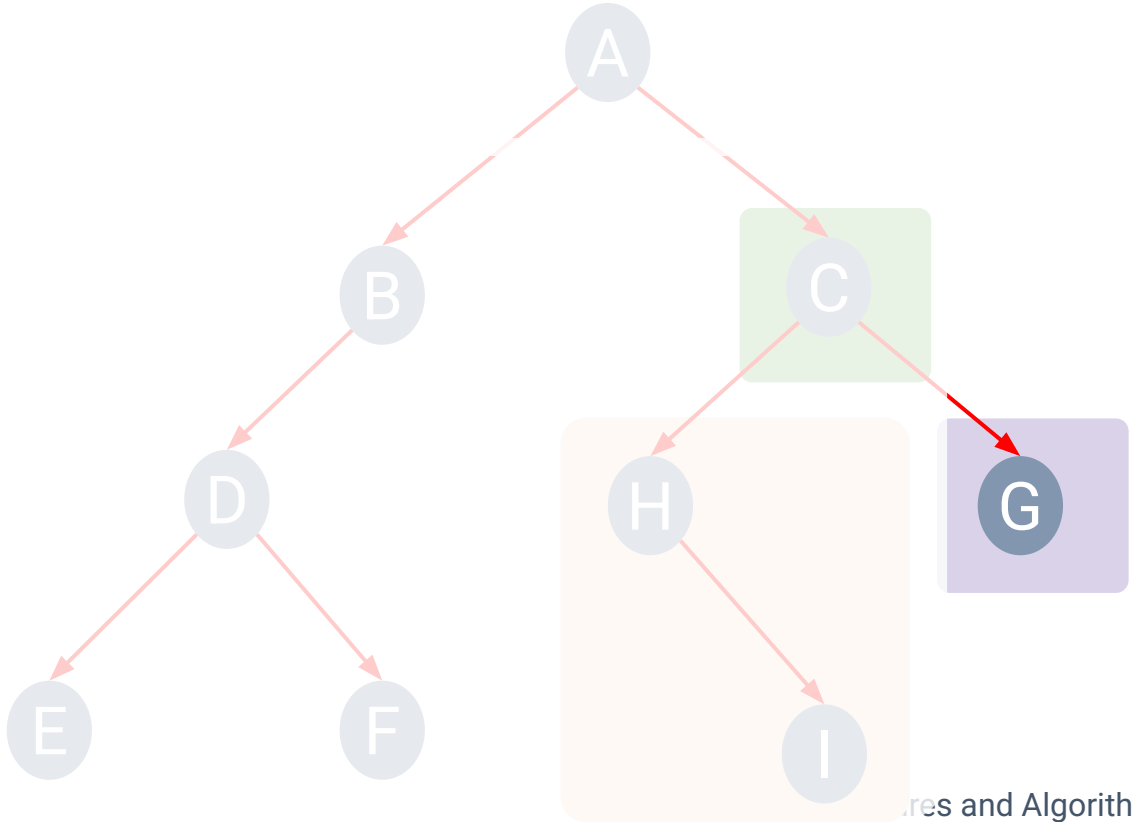
Preorder Traversal

- **Preorder Traversal**

N **L** **R**

- **Visited nodes**

A B D E F
C H I G



Postorder Traversal

- Left subtree -> Right subtree -> root

Algorithm postOrder(root):

if (root is not null):

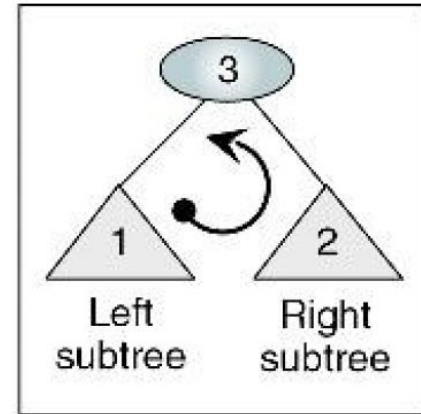
 postOrder(leftSubTree)

 postOrder(rightSubtree)

 process(root)

end if

end postOrder

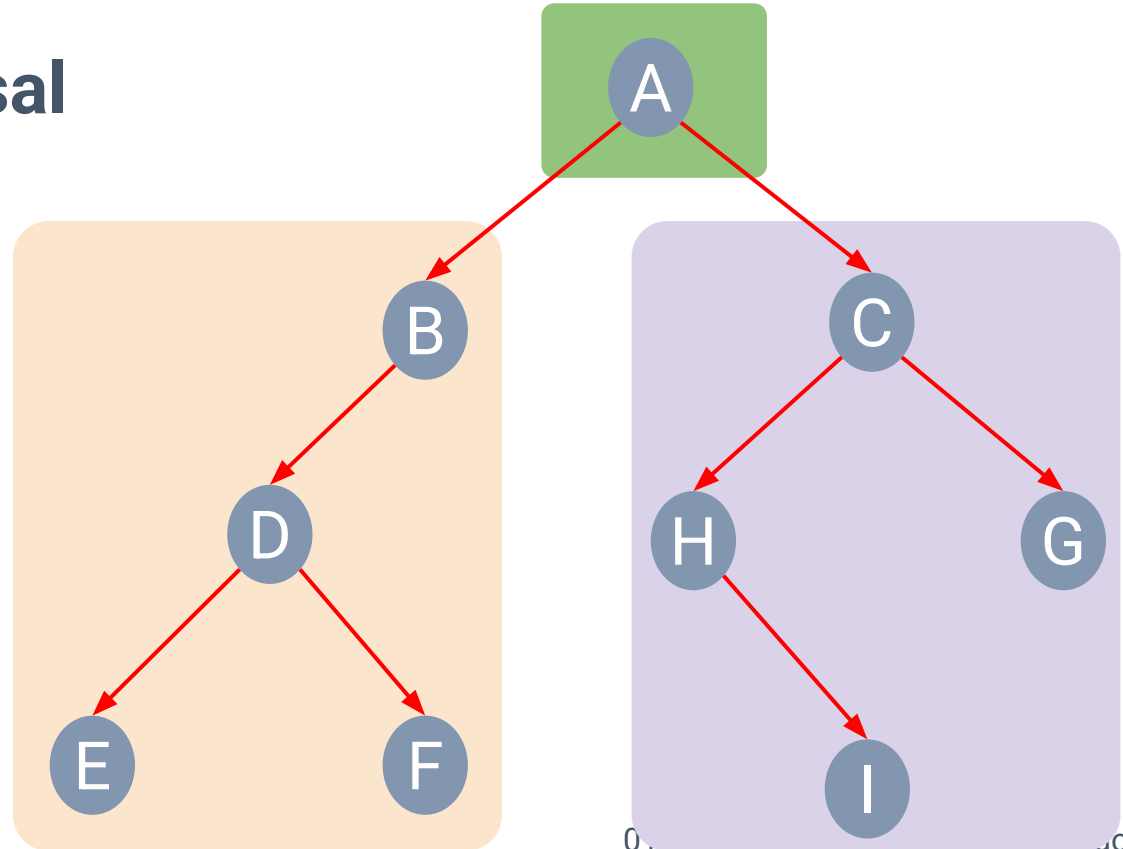


Postorder Traversal

- **Postorder Traversal**

L R N

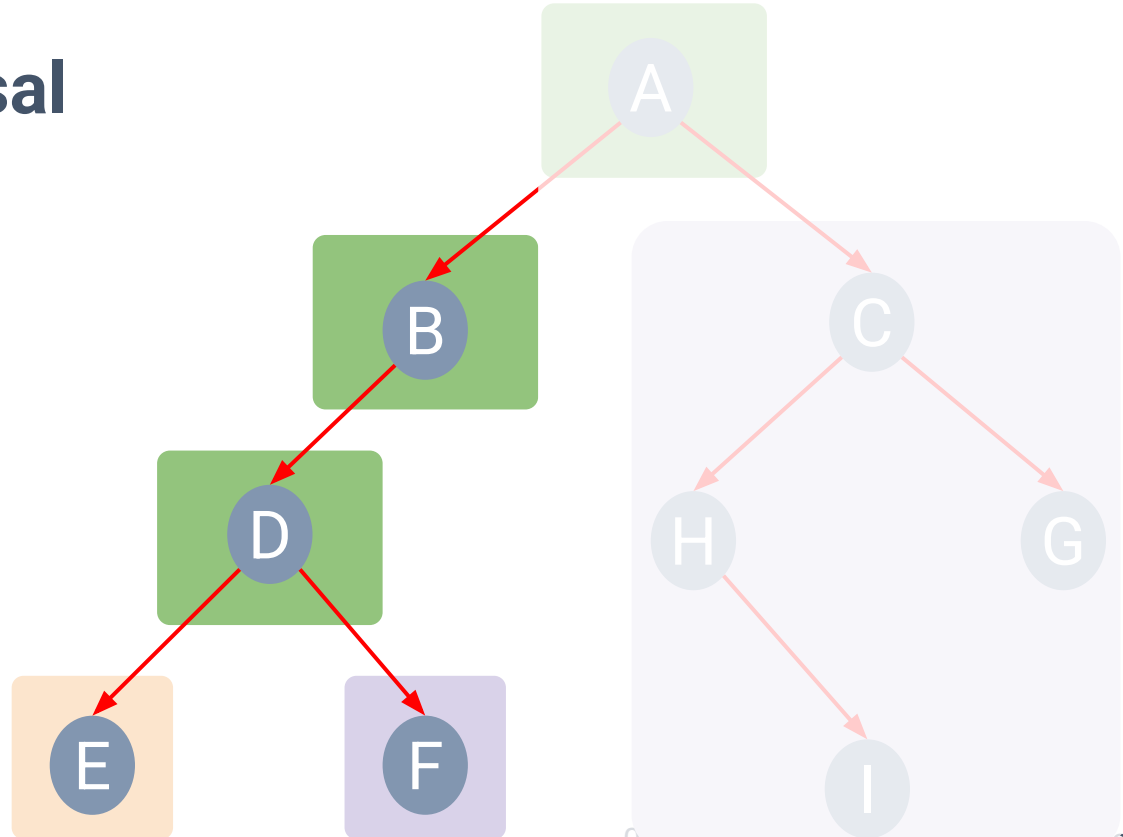
- **Visited nodes**



The logo consists of three rounded squares arranged horizontally. The first square is orange and contains a dark blue letter 'L'. The second square is light purple and contains a dark blue letter 'R'. The third square is green and contains a dark blue letter 'N'.

The logo consists of three rounded squares arranged horizontally. The first square is orange and contains a dark blue letter 'L'. The second square is light purple and contains a dark blue letter 'R'. The third square is green and contains a dark blue letter 'N'.

E F



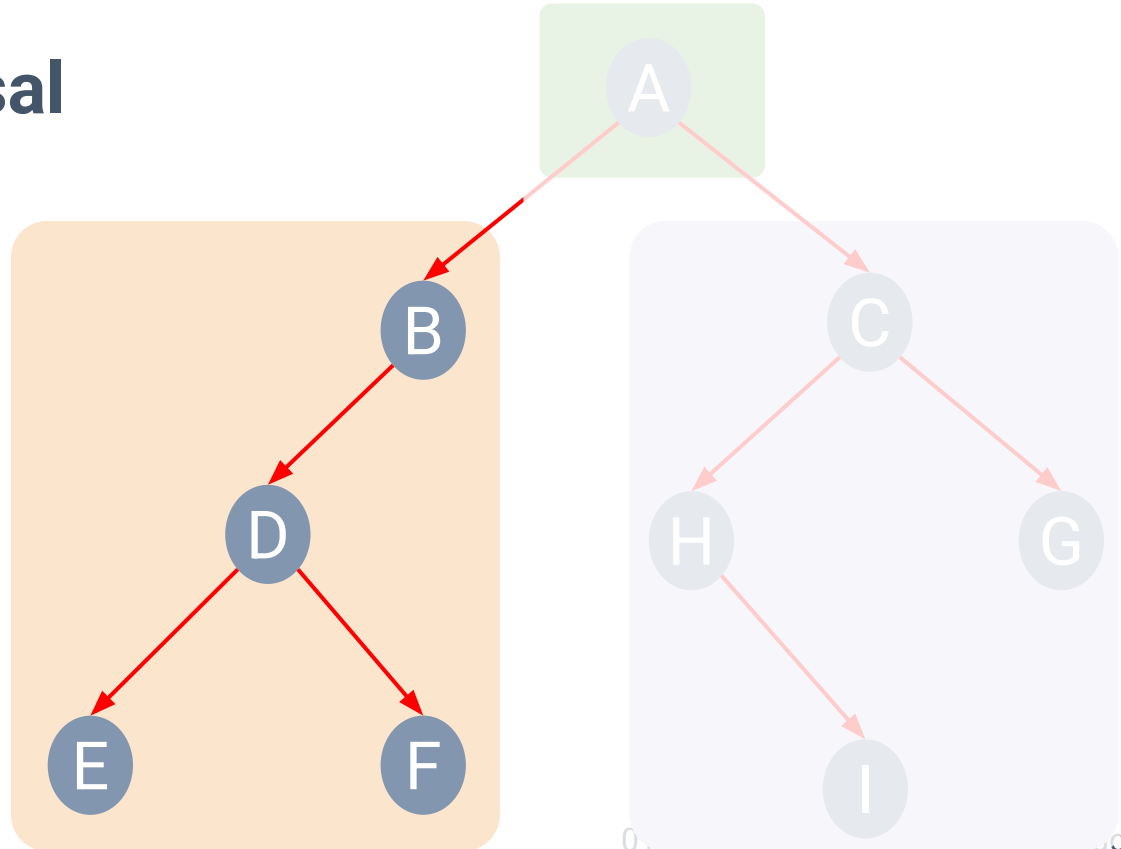
Postorder Traversal

• Postorder Traversal

L R N

• Visited nodes

E F D B



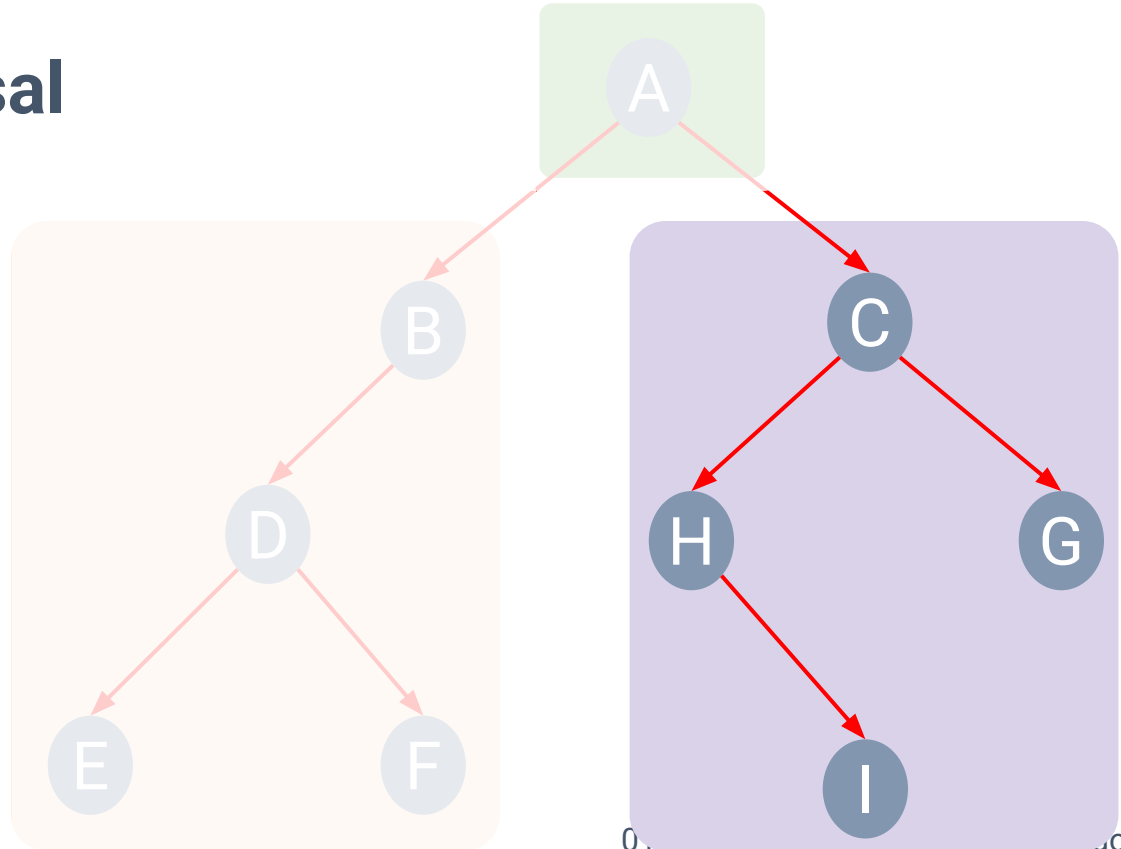
Postorder Traversal

• Postorder Traversal

L R N

• Visited nodes

E	F	D	B
I	H	G	C

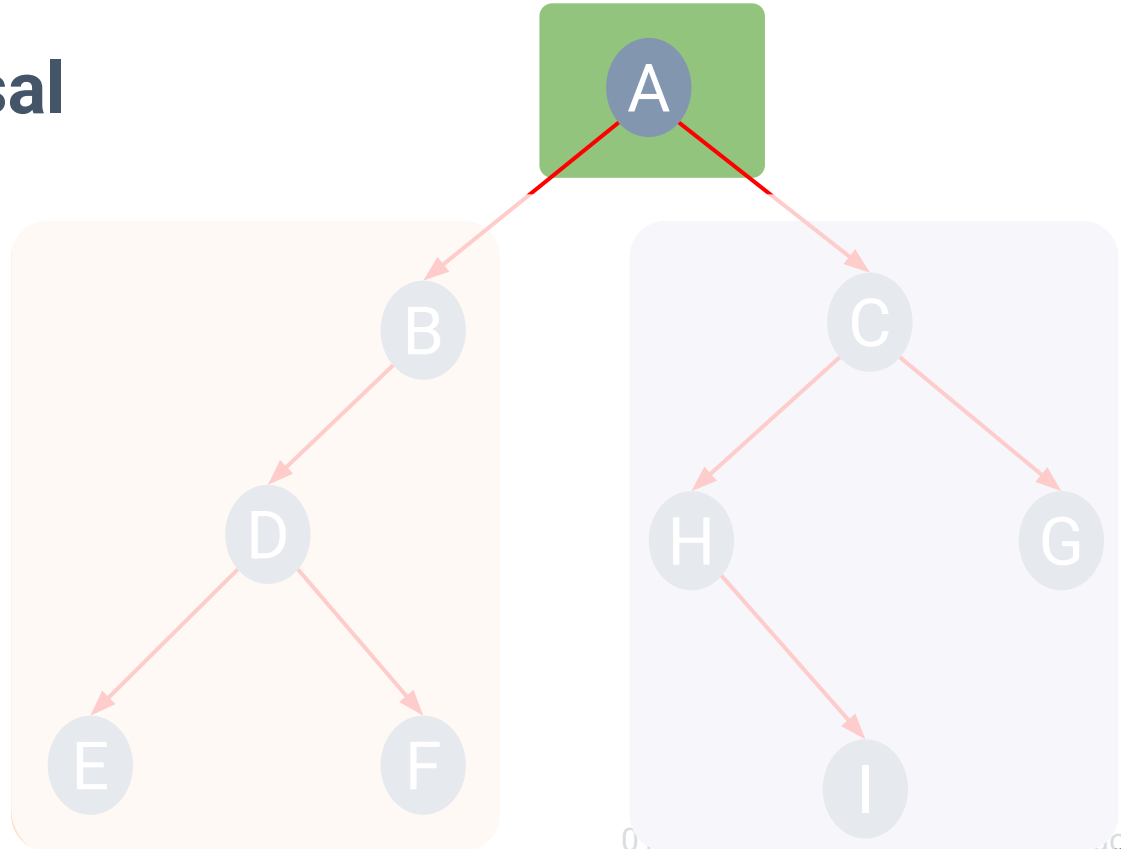
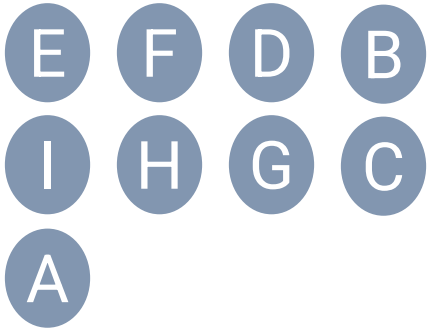


Postorder Traversal

• Postorder Traversal

L R N

• Visited nodes



Inorder Traversal

- Left subtree -> root -> Right subtree

Algorithm inOrder(root):

if (root is not null):

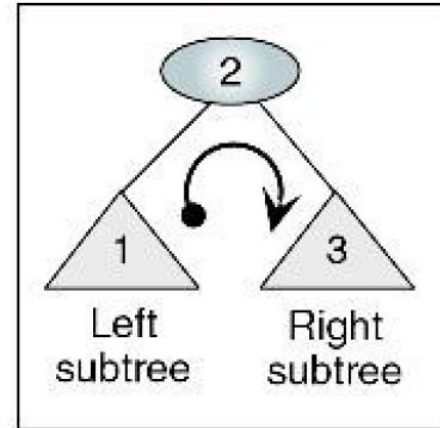
 inOrder(leftSubTree)

 process(root)

 inOrder(rightSubtree)

end if

end inOrder

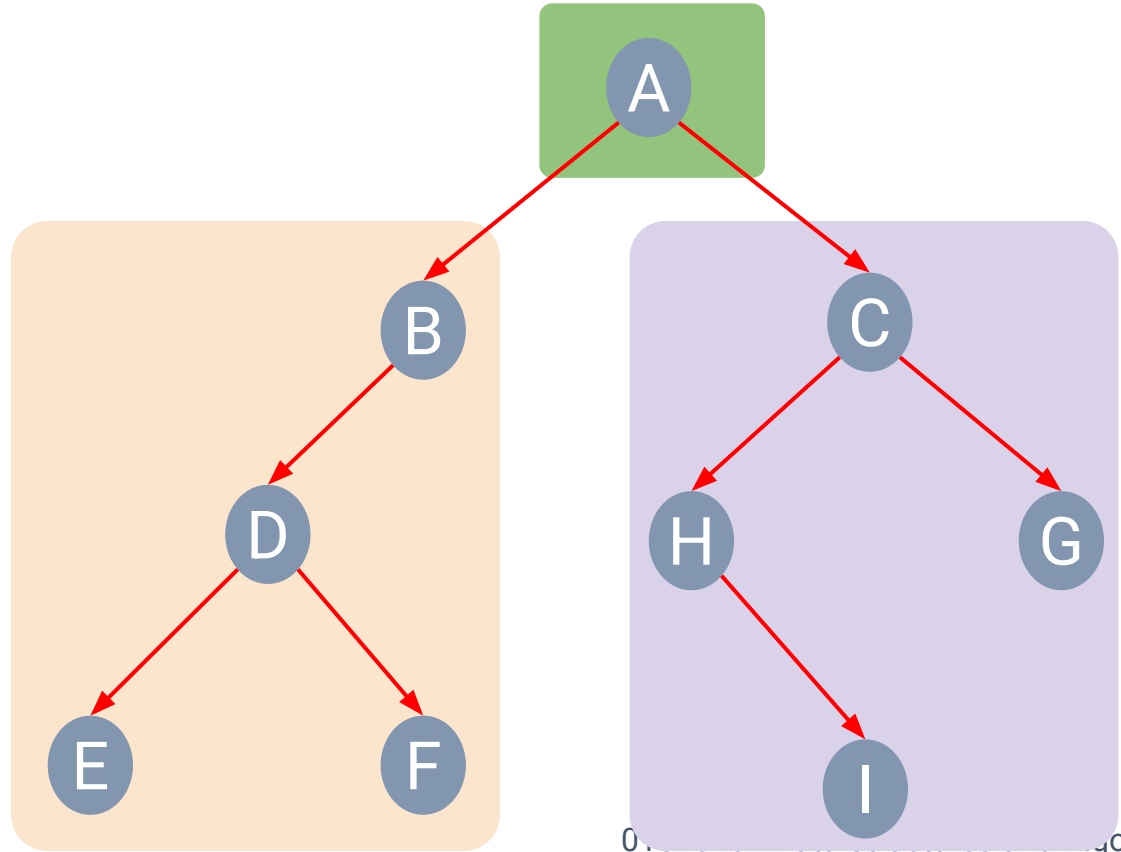


Inorder Traversal

- Inorder Traversal

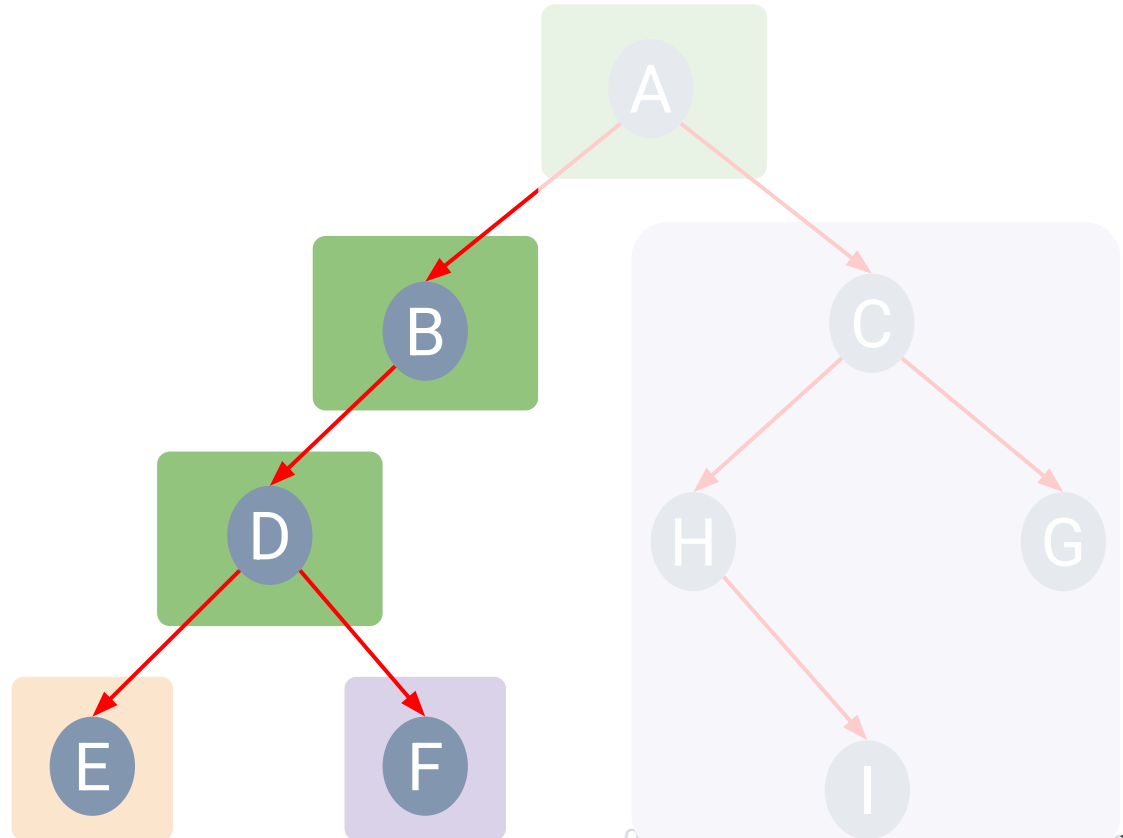
L N R

- Visited nodes



Three colored squares are arranged horizontally. The first square is orange and contains the letter 'L'. The second square is green and contains the letter 'N'. The third square is purple and contains the letter 'R'.

E D F



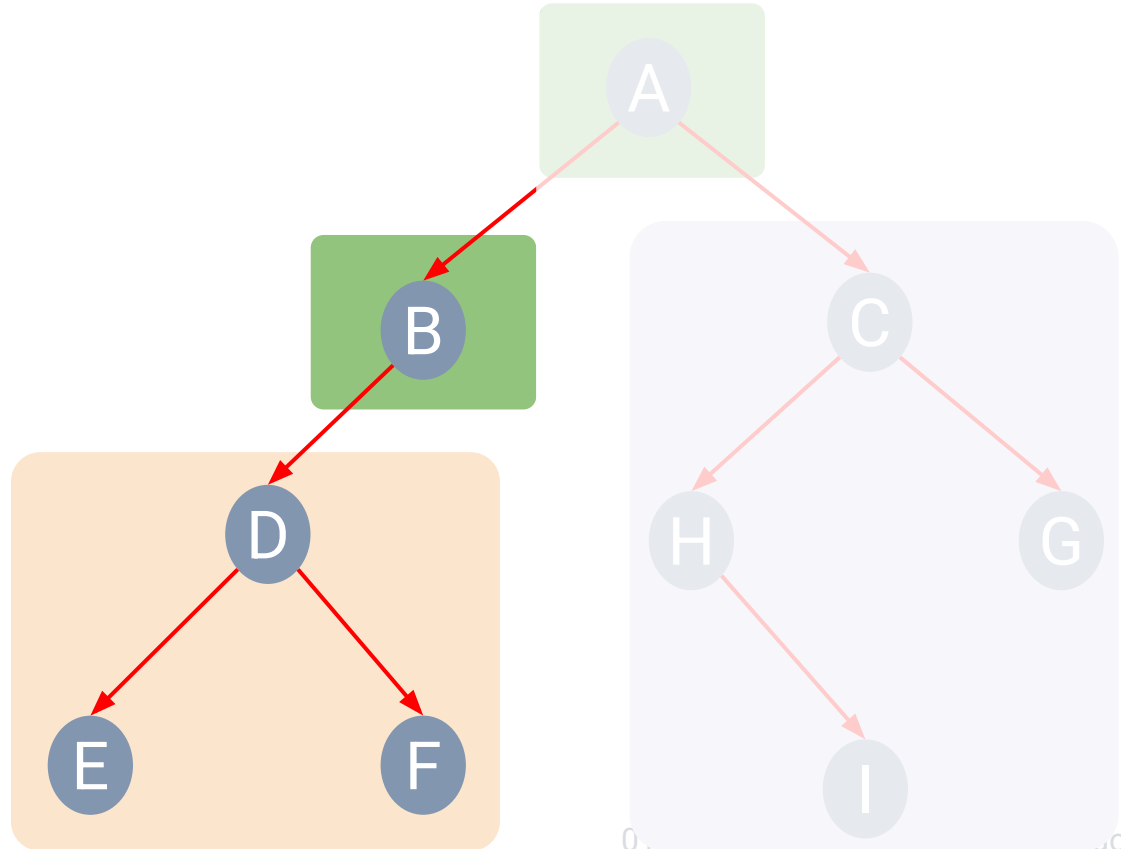
Inorder Traversal

- Inorder Traversal

L N R

- Visited nodes

E D F B



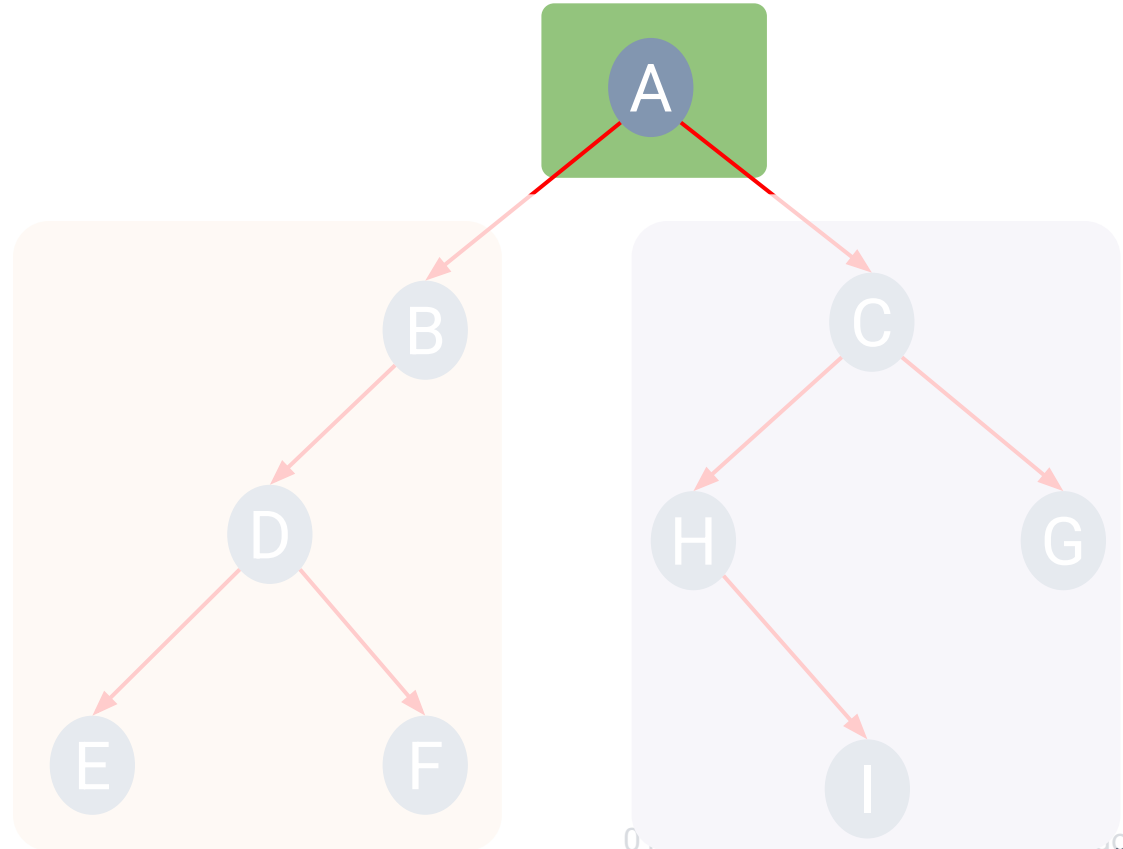
Inorder Traversal

- Inorder Traversal

L N R

- Visited nodes

E D F B A



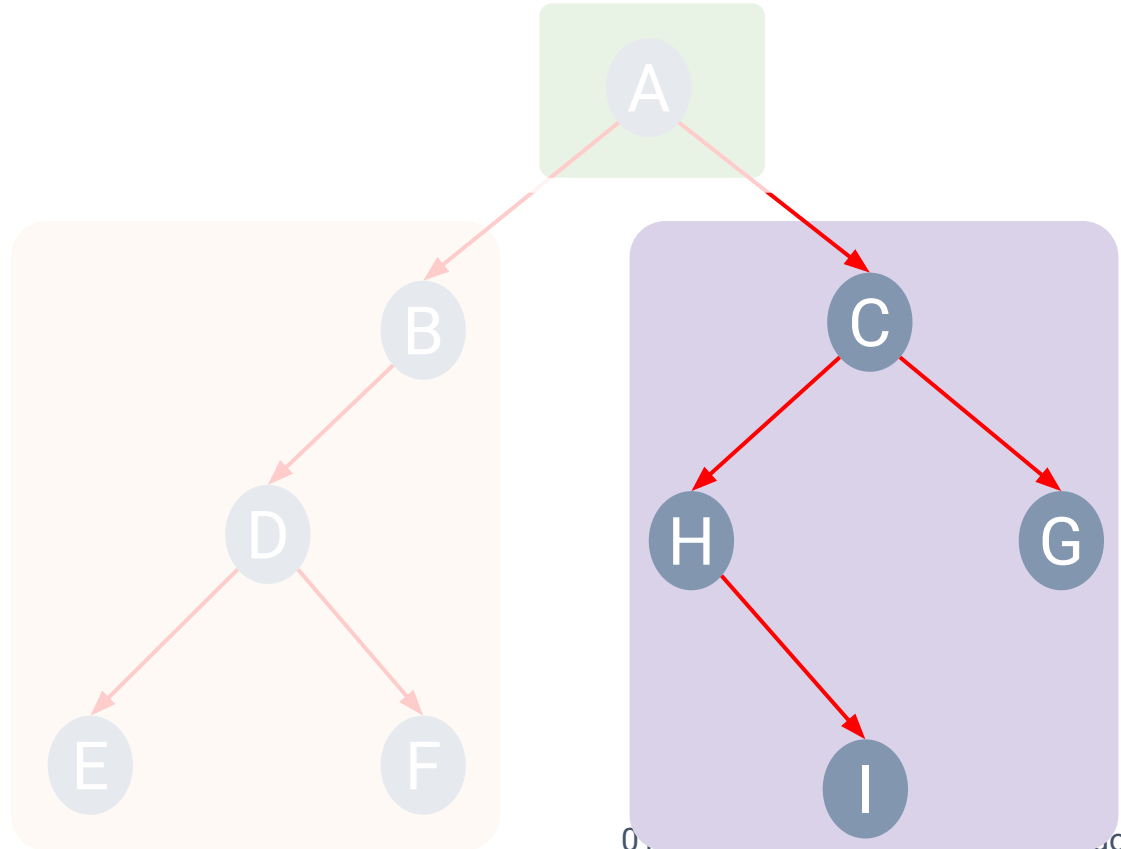
Inorder Traversal

- Inorder Traversal

L N R

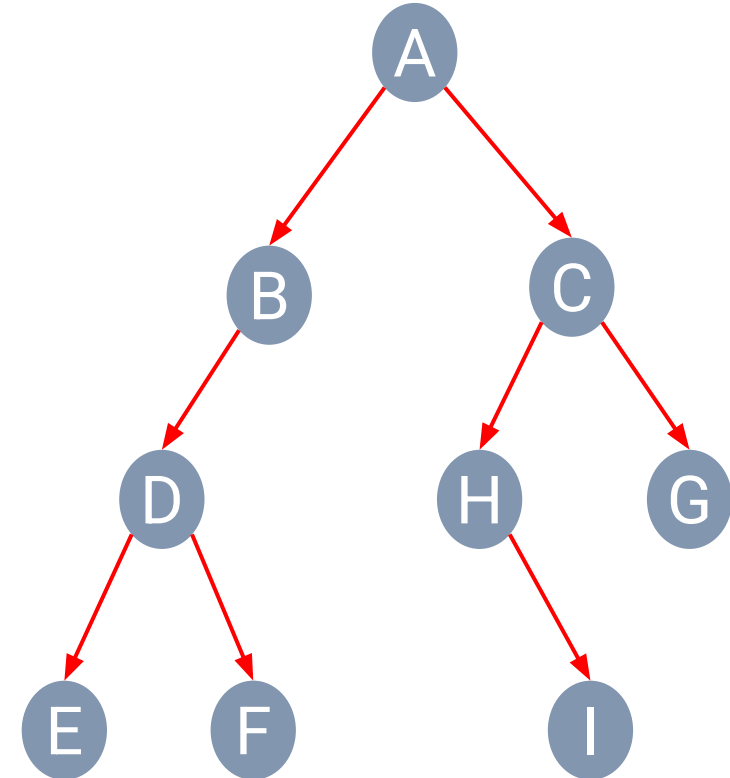
- Visited nodes

E D F B A
H I C G



Depth-first Traversal

Traversal of tree: $O(?)$



• **Preorder** Traversal

N L R

A B D E F C H I G

• **Inorder** Traversal

L N R

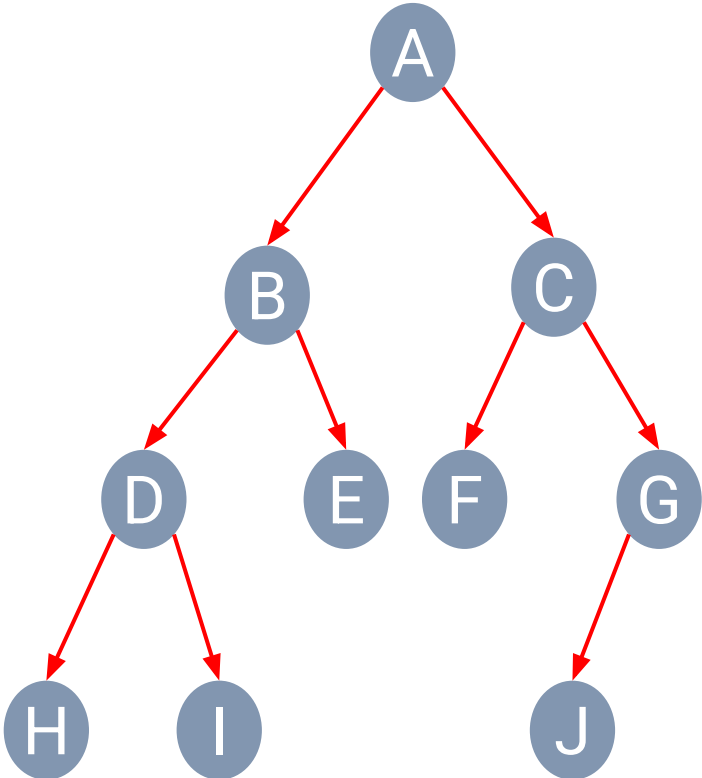
E D F B A H I C G

• **Postorder** Traversal

L R N

E F D B I H G C A

Depth-first Traversal Exercise 1



• **Pre**order Traversal

N **L** **R**

• **In**order Traversal

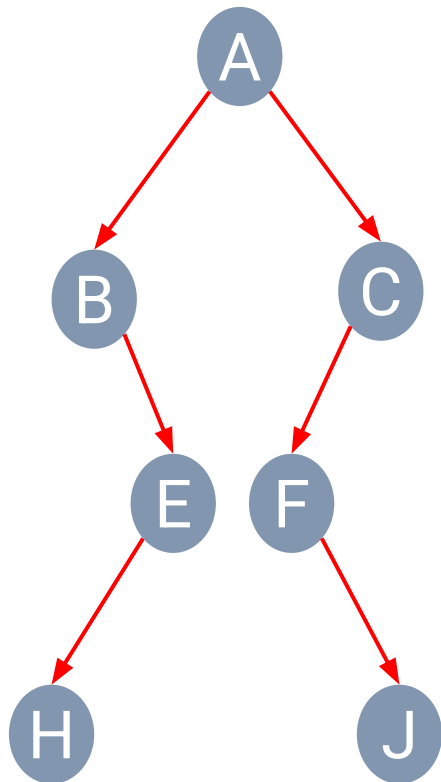
L **N** **R**

• **Post**order Traversal

L **R** **N**



Depth-first Traversal Exercise 2



• **Pre**order Traversal

N L R

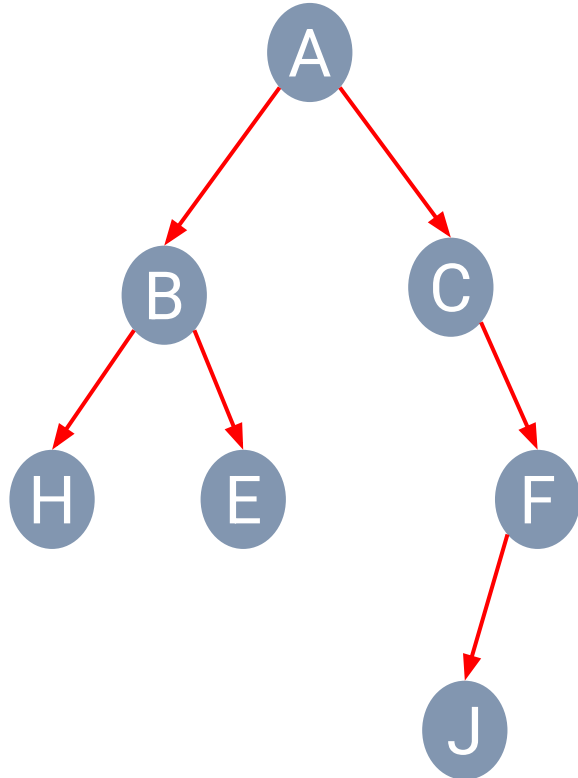
• **In**order Traversal

L N R

• **Post**order Traversal

L R N

Depth-first Traversal Exercise 3



• **Pre**order Traversal

N L R

• **In**order Traversal

L N R

• **Post**order Traversal

L R N



Quiz



1. Find root of Binary Tree from a given traversal path:
 - a. Tree with postorder traversal: FCBDBG
 - b. Tree with preorder traversal: IBCDFEN
 - c. Tree with inorder traversal: CBIDFGE



Quiz



2. Draw a Binary Tree from a given traversal path:
 - a. Preorder: JCBADFEGIH
 - b. Inorder: ABCEDFJGIH



Quiz

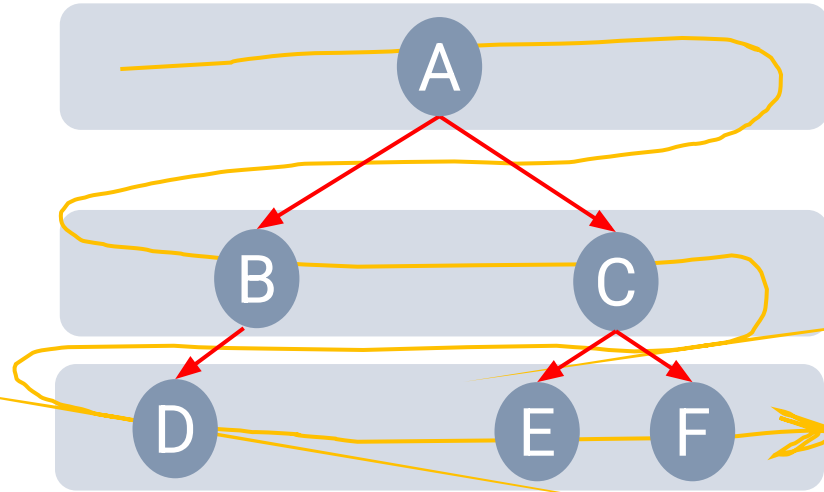


3. Draw a Nearly Complete Binary Tree from a given breadth-first traversal path:

a. JCBADEFIG



Breadth-first Traversal



• Visited nodes

- Visit all the nodes at depth d before moving to depth $d+1$.